

A macro class for grey-deleterium

Design explanation, 1 September 2026. Companion to `macros-v2-guide.md` Part 2, written out properly because you asked for the mechanism rather than the decision points.

Assumes: you have built Part 1, so `spin` is two arguments and the drivetrain holds its own abort source. **Assumes:** the N-tasks shape, one parked task per macro, with a write token per resource.

Where this sits: after the current build. The five leftovers and the robot test still come first, and the `<atomic>` include is still the one line standing between you and a compile. This document is reading, not a queue item.

Done-when for reading it: you can say, in one sentence each, what the class owns, what it refuses to own, and why `exchange` exists. That is the whole deliverable.

The gist

A macro class is `macro_running`, `macro_task` and `macro_routine()` from your `main.cpp` folded into one object, so that a second macro costs one line instead of a second copy of all three.

Start from what you already wrote

Open `main.cpp` and look at what one macro currently takes. There is a flag at file scope, `macro_running`. There is a free function, `macro_worker`, containing the parking loop. There is a second free function, `macro_routine`, containing the actual motion. There is a static `pros::Task` inside `opcontrol`. And there is a branch in the driver loop that reads the flag and decides between claiming and driving.

That is five separate things, none of which know they belong together, all of them at file scope. Now count what a second macro costs you: another flag, another worker with an identical parking loop, another routine, another task, another branch. Four of those five are pure duplication. The routine is the only part that differs, which is a good sign that the routine is the parameter and everything else is the class.

The anchor: one key, several people

Picture a workshop with one door and one key hanging on a hook outside.

Several people want in. Each has their own workbench and their own job to do, and each is sitting outside waiting to be called. To get in, you take the key off the hook. If the hook is empty, someone is already inside and you go back to waiting. When you finish, you walk out and hang the key back up.

The map, part for part:

IN THE WORKSHOP	IN YOUR CODE
the workshop	the drivetrain, the shared resource
the key on its hook	<code>std::atomic<bool> drivetrain_owned</code>
taking the key	<code>ownership->exchange(true)</code>
a person with a workbench	one <code>macro</code> object with its own parked task
hanging the key back	the worker's last act, after the routine returns
the fire alarm	<code>abort_requested</code> , which is not the key

The fire alarm is worth pausing on. It is a separate thing from the key, it tells whoever is inside to leave now, and after Part 1 it is wired into the drivetrain rather than passed around by hand. A macro claims the key. It does not sound the alarm and it does not silence it.

Where the analogy breaks, and it breaks exactly where the interesting part is. With a real hook, you look at it, see a key, and reach for it. Two people can both look before either reaches. Your current `if (!macro_running) macro_running = true;` has precisely that shape: a read, then a write, with a gap in between that a second claimant can walk through. `exchange` is the motion a physical hook cannot give you, a single indivisible look-and-take. That is the one genuinely new idea in this whole document.

What the class owns

Four members, and the reason for each.

`std::function<void()> routine` is the part that differs between macros, so it is the constructor parameter. Any callable fits: a free function like your `macro_routine`, a lambda, a functor. You have been feeding callables to `pros::Task` all along without naming it.

`std::atomic<bool>* ownership` is a pointer to the resource's key, not the key itself. The key belongs to the drivetrain and is shared by every claimant, including the driver loop, so the macro is handed a pointer to it at construction. This is the same move Part 1 made with the abort source, one level up. Notice that and both halves of the macros-v2 guide collapse into one idea: bind the dependency once at construction, and the thing being built never names the global.

`pros::Task worker` is this macro's own task, parked on `notify_take` and doing nothing until someone notifies it.

`std::atomic<bool> running` is the member you might not need. Ask yourself what question it answers that `*ownership` does not. With one shared resource, “is this macro running” and “is anyone writing to the drivetrain” are the same question, and the guard in `opcontrol` wants the second one. Keep `running` only if something wants to know *which* macro is going, for an LCD line or to refuse restarting the same macro. Deleting a member is a real answer here.

What the class must not own: the token itself, and the abort flag. Both belong to the resource.

Two atomics, two jobs

They are easy to blur, so name them apart and keep them apart:

- `drivetrain_owned` answers “who is allowed to write”. Claimed, released, one per resource.
- `abort_requested` answers “please stop what you are doing”. After Part 1 it lives inside the drivetrain and every motion checks it.

A macro touches the first. The drivetrain reads the second.

The skeleton

Finished code is not what you asked for and not what you get. The exception I am taking here is narrow and it is the same one as the abort-flag skeleton on 30 August: the difficulty is the shape and the declaration order, not any individual line. Every body below is something you have already written. So the shape is given and the bodies are holes.

```

// macro.hpp
#pragma once                                // the thing drivetrain.hpp still lacks

#include "pros/rtos.hpp"
#include <atomic>                            // include what you use
#include <functional>

class macro
{
    private:
        std::function<void()> routine;
        std::atomic<bool>* ownership;
        std::atomic<bool> running{false};
        pros::Task worker; // declared LAST, deliberately

        void worker_loop();

    public:
        macro(std::function<void()> routine_to_run,
              std::atomic<bool>& owned_resource);

        bool start();
        bool is_running() const;
};

```

```

// macro.cpp
#include "macro.hpp"

macro::macro(std::function<void()> routine_to_run,
             std::atomic<bool>& owned_resource)
: routine(routine_to_run),
  ownership(&owned_resource),
  // HOLE 1: why can this member not be default-constructed
  //         and assigned in the constructor body instead?
  worker([this]{ worker_loop(); })
{}

void macro::worker_loop()
{
    while (true)
    {
        pros::Task::notify_take(true, TIMEOUT_MAX);

        // HOLE 2: the routine call, and the epilogue.
        // You already know the epilogue is two writes. The only
        // question is their order, and it is the same single-writer
    }
}

```

```

        // argument you made yourself on 31 August.
    }
}

bool macro::start()
{
    // atomically: set it true, and hand me back what it was
    if (ownership->exchange(true)) return false;

    // HOLE 3: three lines. One of them is a decision, not a line.
    // See "the seam" below.
}

bool macro::is_running() const { /* HOLE 4 */ }

```

That shape compiles. I built it against stubs carrying the real PROS signatures out of your vendored `include/pros/rtos.hpp`, with `-Wall -Wextra`, and it is clean.

Four traps

pros::Task has no default constructor. You have met this exact problem before and you wrote the comment yourself, in `drivetrain.cpp`, about `pros::MotorGroup`. Same fix: the member goes in the initializer list, not the constructor body. The consequence is bigger than the syntax, though. Constructing the task *starts* it, so the moment that initializer list reaches `worker`, the lambda is live code capturing a `this` whose other members may not exist yet.

Declaration order is initialization order, so declare the task last. Members initialize in the order they are declared in the class, not the order you write them in the initializer list. If `worker` were declared first, its lambda could touch `routine` before `routine` was constructed. Your worker's first act is parking on `notify_take`, which happens to make this safe, but do not build on that. Declare it last and the order is correct by construction rather than by luck. Search: *member initialization order*, *this escaping the constructor*.

Check-then-set has a hole in it. `if (!owned) owned = true;` is two operations, and two `start()` calls in the same instant can both pass the read before either performs the write. Both then believe they own the drivetrain, which is the judder bug with extra steps. `exchange(true)` writes and returns the previous value as one indivisible operation, so if it hands back `true` you already lost and you have changed nothing you need to undo. The name for this is test-and-set and it is the seed of every lock ever built. Search: `std::atomic::exchange`, *test-and-set*.

A macro cannot live in a std::vector. `std::atomic` is neither copyable nor movable, which makes `macro` neither copyable nor movable. `std::vector<macro> table;` compiles fine, because a vector only instantiates what you use. It breaks the moment you fill it, and the error you get first is this:

```

alloc_traits.h:518: error: no matching function for call to
                    'construct_at(macro*&, macro)'
stl_construct.h:96: error: use of deleted function 'macro::macro(macro&&)'
macro.hpp:6:        error: use of deleted function
                    'std::atomic<bool>::atomic(const std::atomic<bool>&)'

```

Read that bottom-up. The third line is the cause and the first two are the cascade, the same reading discipline as the `<atomic>` error in your header today. `emplace_back` does not save you, because a vector must be able to move its elements when it grows. Build the button table out of pointers to macros that live at file scope, `std::array<macro*, 2>`, and it works.

The seam you have to decide

Part 1 moved `abort_requested` inside the drivetrain, where the macro cannot see it. But `start()` has to clear that flag before it notifies, or a macro started right after an abort will check the flag on its first loop iteration and stop instantly. It will look like the button did nothing.

Two ways out, and this one is yours:

Give the drivetrain a `clear_abort()` and have `main.cpp` call it before `start()`. The macro stays ignorant of aborts, which is clean, but it puts an ordering requirement at every call site, and forgetting it is silent.

Or inject the abort flag into the macro as well, alongside the token, and let `start()` clear it. One more constructor parameter and one more thing the class knows about, but the clearing can never be forgotten because it lives next to the claim.

I would take the second, because “cannot be forgotten” beats “clean” for something that fails silently, but the argument for the first is real and you should make the call.

What `opcontrol` becomes

The driver loop stops asking “is my one macro running” and starts asking “does anything own the drivetrain”. That is the guard. Under it, a button-to-macro table, and the watchdog branch that sets the abort does not move at all.

Roughly: if the drivetrain is owned, watch for the abort button and nothing else. If it is not owned, scan the macro buttons, and if none of them fired, read the sticks and drive. Your `continue` after a successful claim still earns its keep, for the same reason it did before.

Define done before the first line

Not “the class works”. Something you can stand in front of the robot and check:

Two macros are bound to two buttons. A runs the first, X runs the second. Pressing X while the first is running does nothing at all, no judder, no second claimant. B stops whichever is running within a tick, and its remaining legs collapse to instant stops. The sticks are dead for the whole duration of any macro and live the instant it ends. Driver control resumes without restarting the program, twice in a row.

If that passes, the class is finished and you stop.

Names to look up

IDEA	SEARCH TERM
handing an object its dependencies at construction	constructor injection
a routine stored as a value	<code>std::function</code>
why the task member goes last	member initialization order
the lambda that is live before the constructor returns	<code>this</code> escaping the constructor
claiming a shared resource without a race	<code>std::atomic::exchange</code> , test-and-set
why <code>macro</code> will not go in a vector	move semantics, deleted move constructor
the arbitration problem in general	<i>Game Programming Patterns</i> , “State”

Keep this

```
one macro object = routine + its own parked task + a pointer to the resource's key
the key is per resource and injected; the macro never declares it
two atomics, two jobs: owned = who may write, abort = please stop
declare the pros::Task member LAST; it starts running the instant it is constructed
claim with exchange(true), never with check-then-set
macro is non-movable, so no std::vector<macro>; use an array of pointers
```

Next action

Not this. Add `#include <atomic>` to `drivetrain.hpp` and get the current build compiling.